

Автоматический перезапуск рабочих процессов сервера

В жизненном цикле рабочего процесса БЛ возможен ряд экстремальных ситуаций. Например, рабочий процесс, который достаточно давно работает, успел обслужить большое количество запросов и использовать много памяти. Все эти факторы могли отрицательно сказаться на его работе.

Для того чтобы была возможность управлять подобными ситуациями, в сервере приложений СБИС была разработана система качества обслуживания – **QoS (quality of service)**, которая позволяет повысить уровень отказоустойчивости всего приложения в целом.

Реализация системы защиты работы сервисов возможна благодаря многопроцессной архитектуре. Процедура вывода процесса из обслуживания происходит после завершения ранее запущенных методов, не перезапуская весь пул приложений и не теряя запросы.

Ниже описан список проверок, за соблюдением которых следит система.

Система контроля памяти

Потребление памяти не должно превышать установленный лимит в момент, когда рабочий процесс не выполняет ни одного запроса. Значение лимита устанавливается через ini-файл или в админке облака с помощью конфигурационного параметра [Ядро.Сервер приложений.ПределИспользованияПамяти](#) (значение в Мб).

Если рабочий процесс оказывается в состоянии, когда на нем не выполняется ни одного запроса, но при этом его потребление памяти превышает заданный лимит, он перезапускается.

Данный механизм реализован следующим образом:

1. Рабочий процесс периодически асинхронно опрашивает используемую им память:
 - Для ОС Linux – значение разности между RES и SHR;
 - Для ОС Windows – значение RES.
2. Полученное значение сравнивается с пределом допустимого значения используемой памяти (параметр [Ядро.Сервер приложений.ПределИспользованияПамяти](#)).
3. Если полученное значение превышает предел используемой памяти, в координатор рабочих процессов отправляется сообщение: «Больше не добавлять задач на выполнение». При этом выполнение уже запущенных задач продолжается. Далее возможны два варианта событий:
 - Если значение используемой памяти становится ниже предела, в процесс добавляются новые задачи;
 - Если все запущенные ранее задачи выполнились, но значение потребляемой памяти все равно превышает заданный предел, рабочий процесс завершается.

В отличие от рабочих процессов, координатор нельзя перезапустить, так как это приведет к простоя узла сервера и отказом в обслуживании всем запросам, которые уже поступили. Если координатор превысил ограничение по потреблению памяти (408944640 байт), потребив 418402304 байт резидентной неразделяемой памяти, произошла утечка памяти и необходимо обратиться к ответственному за сервис.



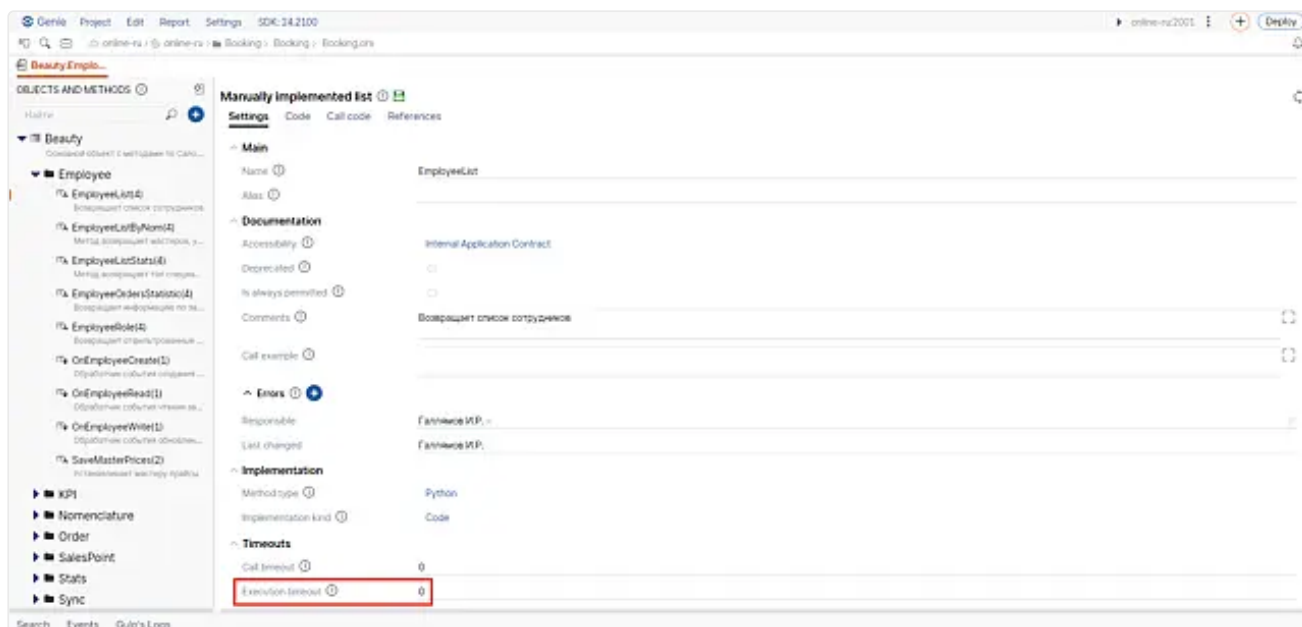
Примечание

При перезапуске рабочих процессов обслуживание запросов не прерывается.

Система контроля времени выполнения методов БЛ

В платформе предусмотрена возможность декларативно ограничить максимальное время работы метода БЛ через Genie. Ограничение накладывается только на внешние вызовы и не распространяется на внутренние, сделанные посредством BLObject.Invoke.

Ограничение задается в Genie с помощью атрибута **Execution timeout** (мс) – жесткого ограничения на время выполнения метода (не путать с **Call timeout**).



Когда в рабочем процессе появляется задача, время выполнения которой превышает значение в атрибуте **Execution timeout** (мс), в [координатор рабочих процессов](#) отправляется сообщение: «Больше не добавлять задач на выполнение».

Далее возможны два сценария событий:

- Просроченные задачи успевают выполниться за время работы задач, у которых срок исполнения еще не истек. В этом случае рабочий процесс продолжает работу;
- Выполнение всех активных в данный момент задач просрочено. В этом случае рабочий процесс завершается.

Примечание

При перезапуске рабочих процессов обслуживание запросов, которые не превысили свое ограничение на время выполнения, не прерывается.

Система контроля утечек транзакций

При работе сервера приложений в ряде случаев на коннектах рабочего процесса сервера приложений к серверу БД остаются незакрытые транзакции, при этом работа метода БЛ уже завершена.

Данная ситуация приводит к накоплению в таких коннектах блокировок на объекты БД, и, впоследствии, к откату всех сделанных изменений. При этом с точки зрения клиентов сервиса, все сделанные ими вызовы завершились успешно, и необходимые изменения были внесены в БД. Такая ситуация может привести к рассинхронизации на сервисах кластера и потере согласованности и целостности данных.

Для минимизации риска возникновения подобных ситуаций, предусмотрена строгая проверка. При обнаружении незакрытых транзакций на момент завершения работы метода рабочий процесс немедленно завершается, не дожидаясь окончания работы уже запущенных в параллельных потоках методов БЛ.

Примечание

При перезапуске рабочих процессов обслуживание всех запросов, выполняемых в данный момент рабочим процессом, прерывается в аварийном режиме.

Система контроля зависания рабочего процесса

В практике работы решений компании были ситуации, при которых рабочий процесс зависал безвозвратно.

Причины подобного поведения могут быть разными. Мы рассматриваем типовой сценарий: сторонняя библиотека вызывает завершение рабочего процесса и при выходе входит в deadlock. Данный сценарий опасен тем, что сервер будет продолжать принимать запросы, но перестанет их обслуживать. В результате образуется очередь.

Для контроля зависания рабочего процесса предусмотрена система проверки, которая регулируется следующими конфигурационными параметрами:

- [Ядро.Сервер приложений.ПроверятьЗависаниеРабочегоПроцесса](#) – отвечает за работу системы проверки зависшего процесса. По умолчанию данный параметр включен.
- [Ядро.Сервер приложений.ДампЗависшегоПроцесса](#) – по завершению зависшего процесса снимается дамп его памяти, что позволяет понять, где возникла ошибка. По умолчанию данный параметр выключен.

Координатор периодически асинхронно отправляет ping на рабочий процесс. Его обслуживает отдельный поток, что позволяет не накладывать ограничений на время работы методов БЛ. Время ожидания ответа – 2 минуты (данный параметр не меняется). Если в течение этого времени ответ не приходит – рабочий процесс признается зависшим и немедленно завершается.

Срабатывание системы Recycling в логах дополнительно может сопровождаться записью предупреждений вида:

Дочерний процесс (pool:X) (pid:Y) (state: 6) был завершен по запросу координатора.